AIMv2

论文：《Multimodal Autoregressive Pre-training of Large Vision Encoders》
代码仓库：apple/ml-aim
模型仓库：apple/aimv2

2025 年初主流 MLLM 使用的 Vision Encoder 都是使用判别式任务（对比学习的方法）进行训练的，如CLIP(2021, OpenAI)、SigLIP、SigLIP2，InternViT会在判别式任务后加上生成式任务。

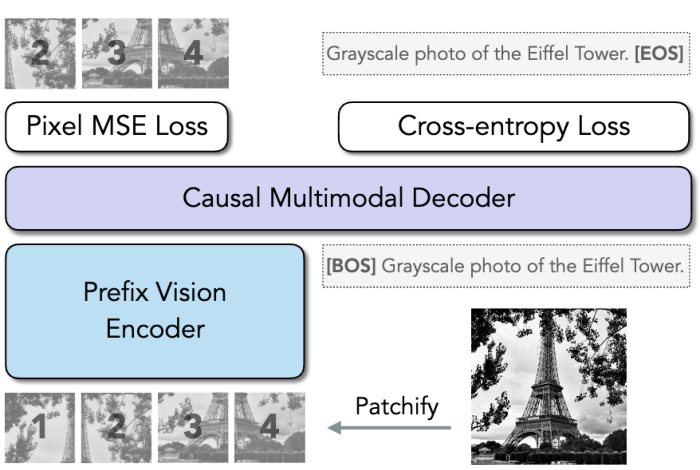
背景：

- 对比学习参数高效，都是进一步训练 scaling 难度大。
- 利用生成式任务进行 CV 预训练效果不好，达到同样效果通常要数倍参数于对比学习。

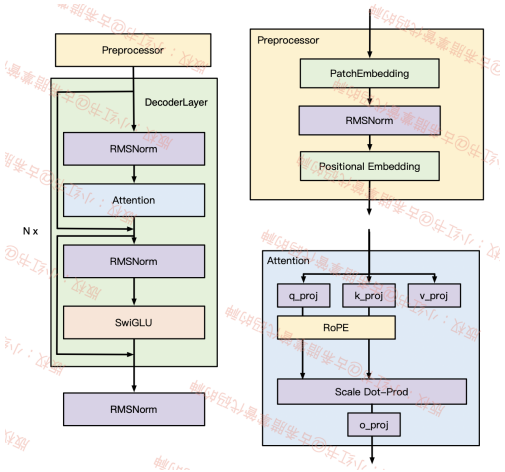
AIMv2 提出了一种**基于多模态自回归的新型 Vision Encoder 预训练框架**，通过统一处理图像与文本进行视觉预训练。

1. 模型架构

模型训练的架构如图（左下）所示，为什么说是“训练的架构”，因为我们预训练的目标模块是 Prefix Vision Encoder，但是需要引入 Casual Multimodal Decoder（语言模型）来进行预训练。而右边是 Prefix Vision Encoder 的内部构造。



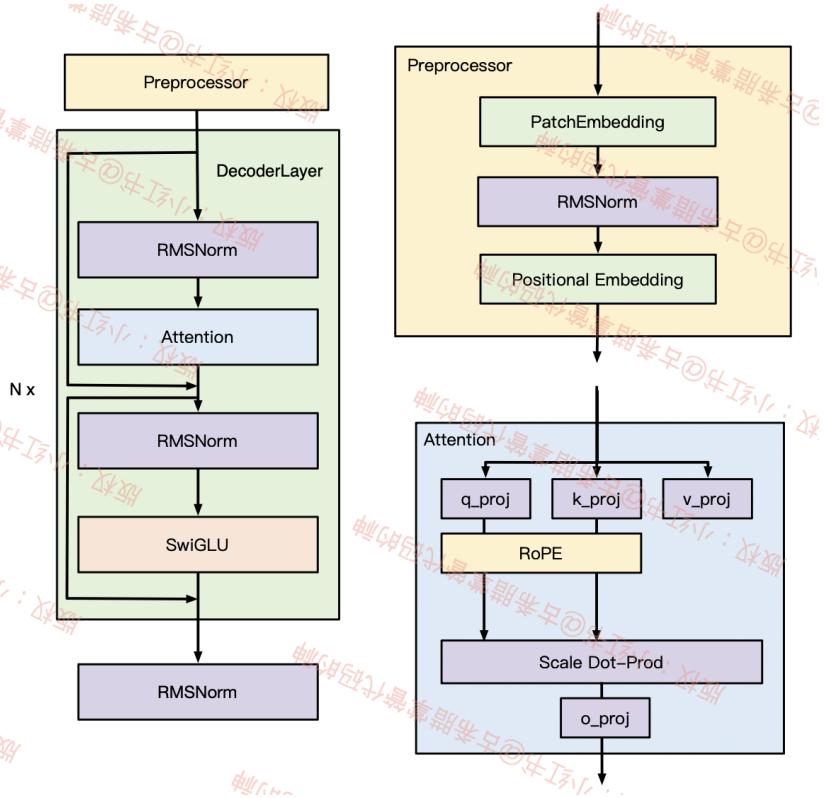
训练架构



Vision Encoder

从左侧训练架构来看，和现在的 MLLM 的架构非常类似，区别在于在视觉部分全部的 feature 都会透出到 LLM 一侧并且计算 MSE Loss，从 LLM 到输出上观察，语言和视觉都是通过 Next Token Prediction 进行训练了，我们将依据左图架构分别介绍各个模块（以及 Loss）。

1.1 Prefix Vision Encoder



Vision Encoder 的架构本质是一个 ViT(2020, Google Research)，但是同 Qwen2.5-VL一样，**AIMv2 的 ViT 使用了更接近语言模型的架构**：RMSNorm + SwiGLU 结构的 FFN

具体来说，归一化层 (Norm) 相比 ViT 的 LayerNorm 替换为了 **RMSNorm**（Root Mean Square Normalization）：

```python
class RMSNorm(nn.Module):
    def __init__(self, dim: int, eps: float = 1e-6):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(dim))
        self.eps = eps

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        output = self._norm(x.float()).type_as(x)
        return output * self.weight

    def extra_repr(self) -> str:
        return f"{tuple(self.weight.shape)}, eps={self.eps}"

    def _norm(self, x: torch.Tensor) -> torch.Tensor:
        return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
```

以及 FFN 使用了 SwiGLU 的结构，**但是相比 Qwen2.5-VL没有启用 bias**：

```python
class AIMv2SwiGLUFFN(nn.Module):
    def __init__(self, config: AIMv2Config):
        super().__init__()
        hidden_features = config.intermediate_size
        in_features = config.hidden_size
        bias = config.use_bias

        self.fc1 = nn.Linear(in_features, hidden_features, bias=bias)
```

```
        self.fc2 = nn.Linear(hidden_features, in_features, bias=bias)
        self.fc3 = nn.Linear(in_features, hidden_features, bias=bias)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = F.silu(self.fc1(x)) * self.fc3(x)
        x = self.fc2(x)
        return x
```

Attention 使用的是标准的 `F.scaled_dot_product_attention`，位置编码使用的是**固定位置编码**（参考ViT的位置编码），不过 AIMv2 也提供了兼容NaViT动态位置编码的版本，可以参考代码。

1.2 Casual Multimodal Decoder

Causal Multimodal Decoder 采用了一个基于 Transformer 架构的自回归多模态解码器，**但是并没有说使用的具体模型来源**。该解码器将图像特征（来自 Vision Transformer 编码器）与文本 token 的嵌入向量进行线性投影后拼接，形成统一的输入序列。图像和文本 token 通过共享的 Transformer 层进行处理，使用因果注意力机制（causal attention mask）确保自回归生成特性。

解码器末端包含两个独立线性投影头：

- 图像头：预测未归一化的图像像素块（使用 MSE 损失）
- 文本头：预测文本 token 的概率分布（使用交叉熵损失）

这种设计允许同时进行图像和文本的生成和 Loss 计算。

1.3 Loss

Vision Encoder 连续 Patch 序列（$x_i \in \mathbb{R}^I$）与文本子词序列（$x_t \in \mathbb{R}^T$）拼接为统一输入序列，通过多模态解码器自回归**分别**生成图像块与文本标记。

损失函数包含两个部分：

- 图像像素回归损失$\mathcal{L}_{img} = \sum_{i=1}^{I} \|\hat{x}_i - x_i\|_2^2$
- 文本交叉熵损失$\mathcal{L}_{text} = -\sum_{t=I+1}^{I+T} \log P(x_t|x_{<t})$

通过比例因子 α 进行加权获得最终的 Loss：

$$\mathcal{L} = \mathcal{L}_{text} + \alpha\mathcal{L}_{text}。$$

> $\alpha = 0.4$达到最优。

Pseudo-code for AIMv2 pre-training

```
# img, cap: input image patches, caption
# I, T: number of patches, number of text tokens
# f_enc, f_dec: Transformer Encoder, Transformer Decoder
def forward(img, cap):
    # prepare targets by shifting the input
    pixel_target, cap_target = shift_left(img), shift_left(cap)

    # sample a prefix len and build attn mask
    attn_mask = build_attn_mask(prefix_len=randint(0, I-1))

    # extract image features
    img_feats = f_enc(I, attn_mask)

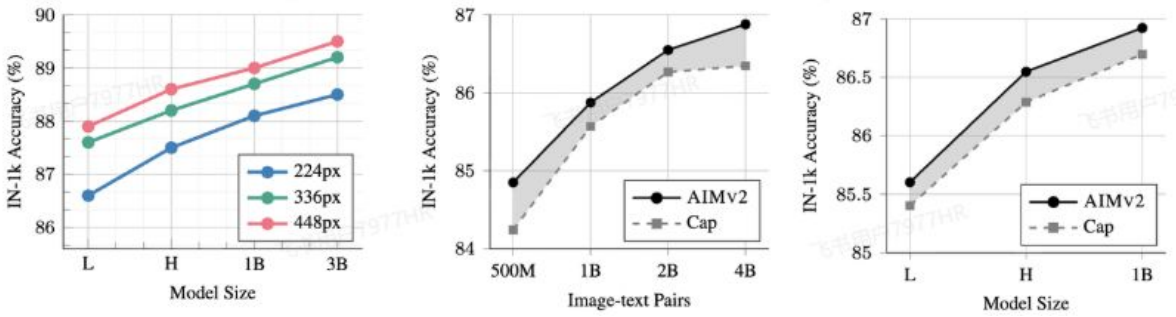
    # concatenate image features and text embedding
    mm_input = concat(img_feats, cap)

    # decode image patches and caption
    mm_out = f_dec(mm_input, is_causal=True)

    # compute loss
    pixel_loss = normalized_mse_loss(mm_out[:I], pixel_target)
    cap_loss = cross_entropy_loss(mm_out[-T:], cap_target)
    loss = cap_loss + alpha * pixel_loss
    return loss
```

2. 预训练 & 后训练

- **预训练**：使用 12B 图像-文本对（224x224），包含公开数据集（DFN-2B、COYO）与私有高质量数据（HQITP），结合合成标注增强多样性。
- **后训练**：使用 2B 量级 336px 和 448px 分辨率图像（与预训练数据同源，不含合成标注），NaViT架构模型进行动态分辨率 Packing 训练。
- **可扩展性**：模型参数范围 300M 至 3B，实验表明数据与模型容量同步扩展时性能持续提升，尤其在计算资源充足时大模型优势显著（如 3B 模型在 IN-1k 达到 89.5% 准确率）。



3. 实验

- **视觉任务**：
  - **图像分类**：冻结主干的 AIM V2-3B 在 IN-1k 达到 89.5% 准确率，超越 MAE、DINOv2 等自监督方法。
  - **目标检测与定位**：在 COCO、LVIS 等基准中，OVD 与 REC 任务上显著优于 DINOv2 与 CLIP 模型。

model	architecture	IN-1k	iNAT-18	Cifar10	Cifar100	Food101	DTD	Pets	Cars	CAM17	PCAM	RxRx1	EuroSAT	fMoW	Infographic
MAE [48]	ViT-2B/14	82.2	70.8	97.5	87.3	93.4	81.2	95.1	94.9	94.4	90.3	7.3	98.2	60.1	50.2
AIMv1 [33]	ViT-H/14	78.5	64.0	97.2	86.8	90.1	80.1	93.0	93.0	94.3	90.0	7.8	98.4	58.3	45.2
	ViT-7B/14	84.0	75.5	98.9	91.8	94.1	85.6	95.4	95.0	94.2	90.5	8.4	98.5	63.5	57.7
DINOv2 [87]	ViT-g/14	87.2	83.0	99.7	95.6	96.0	86.9	96.8	94.9	95.8	90.1	9.0	98.8	65.5	59.4
OAI CLIP [94]	ViT-L/14	85.7	73.5	98.7	89.7	95.4	83.5	96.2	94.5	94.4	89.2	5.7	98.0	62.0	66.9
DFN-CLIP [35]	VIT-L/14	86.5	75.5	99.2	93.2	96.2	85.8	96.3	96.4	95.0	89.8	5.8	98.3	63.1	66.8
	ViT-H/14	86.9	76.4	99.3	93.9	96.3	87.0	96.8	96.7	95.7	90.5	6.1	98.8	63.4	68.1
SigLIP [133]	ViT-L/16	86.5	75.1	98.5	90.4	96.1	86.7	96.7	96.5	93.1	89.5	4.5	98.3	61.7	71.0
	ViT-So400m/14	87.3	77.4	98.8	91.2	96.5	87.7	96.7	96.6	93.3	90.0	4.6	98.6	64.4	72.3
AIMv2	ViT-L/14	86.6	76.0	99.1	92.2	95.7	87.9	96.3	96.3	93.7	89.3	5.6	98.4	60.7	69.0
	ViT-H/14	87.5	77.9	99.3	93.5	96.3	88.2	96.6	96.4	93.3	89.3	5.8	98.5	62.2	70.4
	ViT-1B/14	88.1	79.7	99.4	94.1	96.7	88.4	96.8	96.5	94.2	89.0	6.7	98.8	63.2	71.7
	ViT-3B/14	88.5	81.5	99.5	94.3	96.8	88.9	97.1	96.5	93.5	89.4	7.3	99.0	64.2	72.2
	ViT-3B/14$_{448px}$	89.5	85.9	99.5	94.5	97.4	89.0	97.4	96.7	93.4	89.9	9.5	98.9	66.1	74.8

- **多模态理解**：
  - **指令微调**：结合 LLM（如 Llama 3）进行单阶段微调，在 VQAv2、TextVQA 等问答任务中优于 SigLIP 与 CLIP，且支持高分辨率分块推理（最高 1008px）。
  - **上下文学习**：在 4/8 样本少样本设置下，多模态 ICL 性能超越 MM1 系列模型，验证其与大规模多模态管道的兼容性。

model	architecture	# patches	VQAv2	GQA	OKVQA	TextVQA	DocVQA	InfoVQA	ChartQA	ScienceQA	COCO	TextCaps	NoCaps	SEED	MME$_p$
OpenAI CLIP	ViT-L/14	576	78.0	72.0	60.0	47.5	25.6	21.8	18.5	73.8	94.9	75.3	93.3	70.1	1481
SigLIP	ViT-L/14	576	76.9	70.3	59.3	44.1	16.9	20.7	14.4	74.7	93.0	69.9	92.7	66.8	1416
	ViT-So400M/14	752	77.7	71.0	60.1	47.5	19.2	21.0	14.7	74.9	94.6	70.8	94.5	67.5	1433
DINOv2	VIT-g/14	3034	76.7	72.7	56.9	15.1	8.2	19.7	12.0	69.5	93.4	42.1	89.1	68.9	1423
AIMv2	ViT-L/14	576	79.7	72.5	60.8	53.6	26.6	22.8	19.2	74.1	96.9	81.1	99.9	71.8	1472
	ViT-H/14	576	80.2	72.8	61.3	55.5	27.8	**23.1**	19.9	76.8	99.6	80.7	102.7	72.1	1545
	ViT-1B/14	576	80.5	73.0	61.5	56.8	28.5	22.1	20.5	76.4	98.4	82.6	101.5	72.7	1508
	ViT-3B/14	576	**80.9**	**73.3**	**61.7**	**58.2**	**30.4**	23.0	**22.6**	**77.3**	**100.3**	**83.8**	**102.9**	**72.9**	**1545**